

📌 نبذة عن لغة JavaScript

هي لغة برمجية تم ابتكارها وتطويرها من قبل شركة Netscape لغرض جعل المواقع الإلكترونية متفاعلة وأكثر إستجابة وتفاعلية وحيوية مع الزائر وبالتالي فهي تختلف عن لغات التصميم كلغة HTML ولغة CSS تماماً لأنها تُستعمل لبرمجة بعض الأمور كالنماذج Forms بحيث يرى الزائر نتائجها أمامه وتُنفذ في جهة الزائر Client Side بحيث يصبح الزائر قادر على التفاعل معها بسهولة ورؤية نتائجها وهذا ما كان يستحيل علينا فعله باستخدام لغتي HTML و CSS وسبب اختلاف لغة JavaScript عن لغة HTML و لغة CSS ليس في الاستخدام فقط بل حتى في شكلها ومفاهيمها ونتائجها ففي الوقت الذي تُستعمل به لغة HTML لبناء هيكل المواقع ولغة CSS لتصميم الموقع تجد إن لغة JavaScript بعيدة كلياً عن التصميم فهي تدعم اغلب المفاهيم الموجودة في اللغات القوية مثل لغة C و C++ و C# و Java و Python و VB وهي المفاهيم المشتركة بين هذه اللغات كمفهوم البنية الشرطية والبنية التكرارية والدوال والمصفوفات والكائنات وغيرها من المفاهيم لذا فأنتك تجد لغة JavaScript تُقارب تلك اللغات القوية ومع ذلك لا تصل إليها في القوة بسبب إقتصار إستخدام تلك اللغة على برمجة المواقع الإلكترونية من جهة العميل فقط على عكس اللغات الأخرى القوية التي تُستعمل في كل المجالات وبالتالي فإنك لا تجد مطور مواقع الكترونية لا يعرف لغة JavaScript أو لا يحتاج إلى لغة JavaScript لذا تعال بنا نتعرف على هذه اللغة القوية والجميلة ونتعرف على أبرز مفاهيمها وكيف سنُبرمج بها مواقعنا وكم هي مفيدة وما هي النتائج المُرتبة عليها .

📌 كيفية العمل

إن شيفرة لغة JavaScript تمتاز بإمكانية إقحامها مع شيفرة لغة HTML ولكي نكتب شيفرة لغة JavaScript فإنه لدينا طريقتين لفعل ذلك والطريقة الأولى تتم بإستخدام الوسم الثنائي <script> حيث نكتب بين بداية ونهاية هذا الوسم شيفرة هذه اللغة وتلك الطريقة لديها عدة أشكال وكما تلاحظ :

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Learning js</title>
</head>

<body>
  <script language="javascript">
    /*اكتب هنا شيفرة لغة الجافا سكريبت*/
  </script>
</body>
</html>
```

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Learning js</title>
</head>

<body>
  <script type="text/javascript">
    /*اكتب هُنا شيفرة لغة الجافا سكربت*/
  </script>
</body>
</html>

```

طبعاً تُلاحظ النمط الذي استخدمناه في كتابة شيفرة الجافا سكربت بأسلوبين لا يختلفان عن بعضهما البعض ففي الأسلوب الأول استخدمنا الكلمة language في تحديد لغة البرمجة المراد البرمجة بها واختارنا لغة JavaScript دلالةً على أننا بصدد البرمجة باستخدام هذه اللغة وبالتالي فإن أي شيفرة نكتبها بين بداية ونهاية الوسم الثنائي <script> تكون تابعة لهذه اللغة أما في الأسلوب الثاني فقد استخدمنا الكلمة type بدل language لتحديد لغة البرمجة التي نريد أن نُبرمج بها موقعنا واختارنا لغة JavaScript وهذا يعني أننا بصدد البرمجة باستخدام هذه اللغة وبالتالي فإن أي شيفرة نكتبها بين بداية ونهاية الوسم الثنائي <script> هي خاصة بلغة جافا سكربت كذلك نستطيع أن نكتب شيفرة هذه اللغة بدون استخدام الكلمات type او language في تحديد لغة البرمجة بل نكتفي بكتابة الوسم <script> فقط والمُتصفح سيعرف تلقائياً إن هذه الشيفرة تابعة للغة JavaScript وكما تُلاحظ:

```

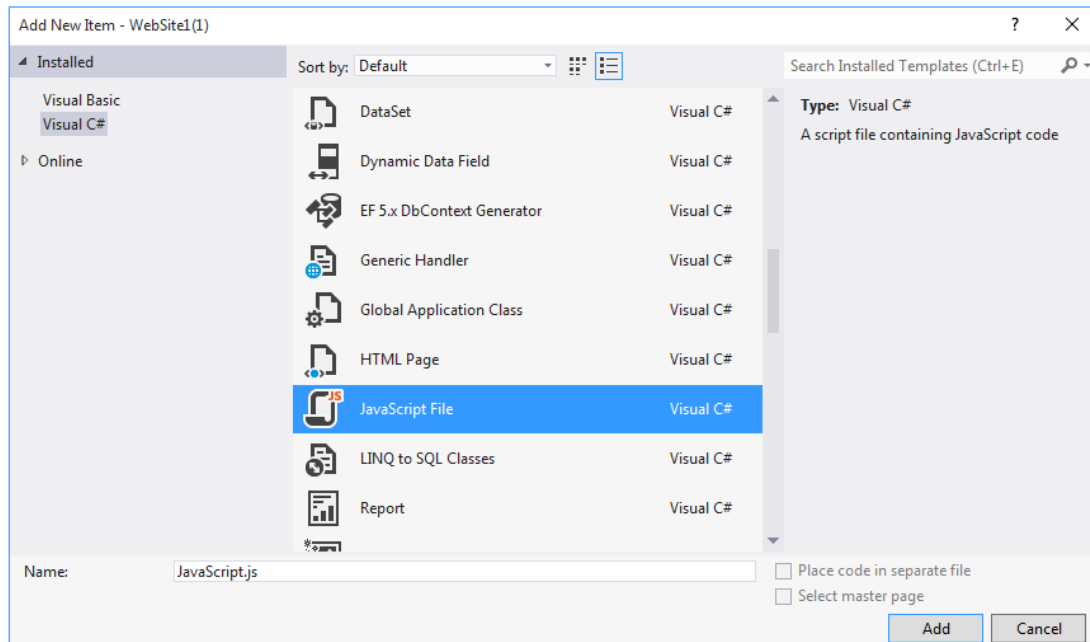
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Learning js</title>
</head>

<body>
  <script>
    .
    .
    //اكتب هُنا شيفرة لغة الجافا سكربت
    .
    .
  </script>
</body>
</html>

```

هذا بالنسبة للطريقة الأولى من طرق كتابة شيفرة الجافا سكربت أما بالنسبة للطريقة الثانية فهي تتم من خلال إنشاء ملف JavaScript أي ملف بامتداد js وبداخل هذا الملف نكتب شيفرة هذه اللغة ونقوم بربطه مع صفحة لغة HTML أي نربطه بالموقع الإلكتروني وهذه الطريقة هي الأفضل لأننا إذا استعملنا الطريقة السابقة بأساليبها المختلفة فإننا سنواجه مشكلة تكرار الشيفرة على مستوى عدة صفحات هذا إذا كان موقعنا الإلكتروني يتكون من عدة صفحات وبالتالي فإننا إذا أردنا استخدام نفس الشيفرة في تلك الصفحات وجب تكرارها على مستوى جميع الصفحات بينما لو قمنا بعزل الشيفرة في ملف خاص فلا يتطلب منا الأمر سوى ربط الملف في كل صفحة وبالتالي نتخلص من مشكلة التكرار ومسألة إنشاء ملف جافا

سكربت في برنامج Visual Studio تتم بنفس الطريقة التي نُشئ بها ملف HTML او ملف CSS إذ كُل ما علينا فعله هو تحديد المُجلد الذي يضم ملفات مشروعك والذي يظهر في النافذة Solution Explorer والضغط على مفتاح الاختصار Ctrl+Shift+A لتظهر لك نافذة إضافة العناصر ومنها حدد ملف JavaScript واضغط على الزر Add لتتم إضافته إلى ملفات المشروع :



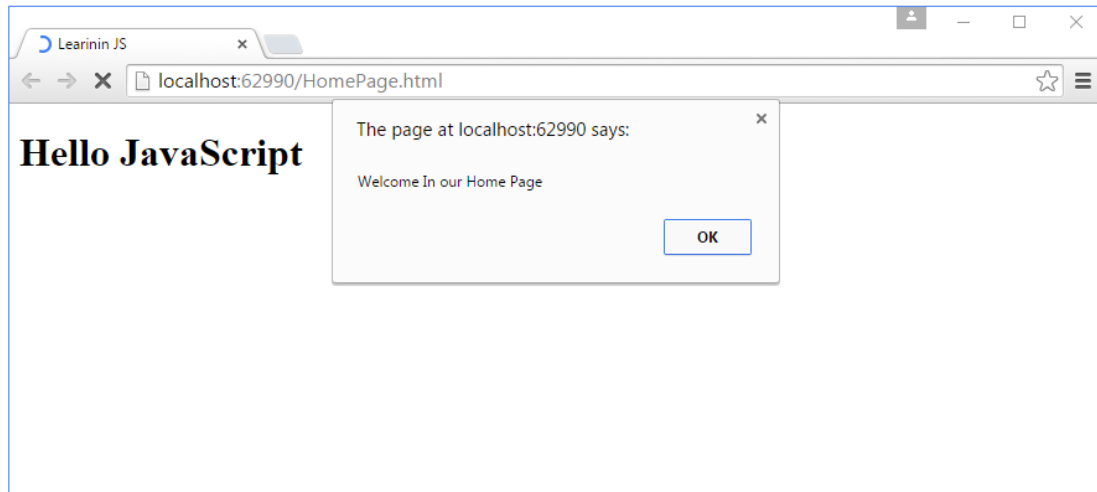
ولكي نربط هذا الملف بصفحة HTML نكتب الشيفرة التالية في منطقة الوسم head :

```
<head>
  <title>Learning js</title>
  <script src="MyJavaScriptFile.js"></script>
</head>
```

وبالتالي فإن أي شيفرة نكتبها في هذا الملف سيتم استعراضها وتنفيذها نتيجة لعملية الربط ويمكن ربط هذا الملف مع أكثر من صفحة HTML ليتم تطبيق الشيفرة التي بداخله على مستوى جميع تلك الصفحات وكذلك يمكن ربط أكثر من ملف على نفس الصفحة او على أكثر من صفحة وبالتالي فإن هذه الطريقة تُعتبر الأمثل بالمُقارنة مع الطريقة السابقة . الآن بعد أن تعرفنا على طُرق كتابة شيفرة لغة الجافا سكربت تعال بنا نكتب شيفرة بسيطة بهذه اللغة نتعرف من خلالها أسلوب وطبيعة هذه اللغة لذا ادخل ملف لغة الجافا سكربت واكتب الشيفرة التالية بداخله :

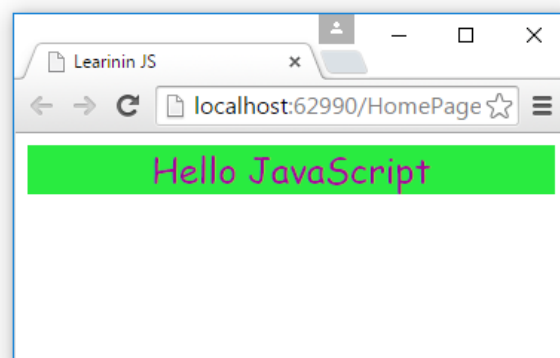
```
document.write("<h1> Hello JavaScript </h1>");
window.alert("Welcome In our Home Page");
```

ولا تنسى ربط هذا الملف بصفحة HTML كما تعلمنا وعند تنفيذ الصفحة في المُتصفح ستحصل على النتيجة التالية في المُتصفح :



لاحظ في السطر الأول من الشيفرة قُمنا بإظهار نص مُعين في المُتصفح بأسلوب لغة الجافا سكربت حيث أستخدمنا الدالة `write` لإظهار النص ونستطيع التعويض عن تلك الدالة بالدالة `writeln` لإظهار النص إلا إن هذه الدالة تقوم بطباعة النص في سطر وتهيئة سطر جديد بحيث يتم إنزال المحتوى الذي يقع أسفل النص المطبوع في سطر جديد على عكس الدالة `write` التي تطبع النص في سطر واحد دون تهيئة سطر جديد أما بالنسبة للسطر الثاني من الشيفرة فكما تلاحظ فمن خلاله قُمنا بإظهار نافذة `ال alert` وهي النافذة الصغيرة التي تتوسط المُتصفح كما تلاحظ في الصورة أعلاه وهذه النافذة تظهر عند تحميل الصفحة ولا يمكن الولوج للصفحة إلا بالضغط على الزر `OK` في هذه النافذة لكي يتم الولوج بك إلى الصفحة وعادةً ما تُستعمل هذه النافذة لإظهار رسالة ترحيبية للزائر عند دخوله للموقع أو لغرض إجبار الزائر على إدخال اسمه والترحيب به على أساس القيمة المُدخلة ولعلك كذلك لاحظت أننا طبقنا وسم خاص بلغة `HTML` على شيفرة لغة `JavaScript` وهو الوسم `<h1>` وهذه هي إحدى ميزات لغة الجافا سكربت إذ تسمح هذه اللغة بإقحام شيفرات لغة `HTML` ولغة `CSS` مع شيفراتها فمثلاً إذا أردنا أن نجعل النص الذي نريد طباعته باستخدام شيفرة الجافا سكربت أن يظهر في منتصف الصفحة باستخدام الوسم `<center>` وهو من أوسمة لغة `HTML` وبنفس الوقت نريد تلوين ذلك النص بلون مُعين مع تغيير نمط النص إلى `Comic Sans MS` وتغيير حجم النص إلى `18pt` وإعطاء النص خلفية بلون أخضر عندها سيكون شكل الشيفرة ونتيجتها كالتالي:

```
document.writeln("<center style='color:#a8079b;font-family:comic Sans MS,impact;font-Size:18pt;background-color:#29eb40;'>Hello JavaScript </center>");
```



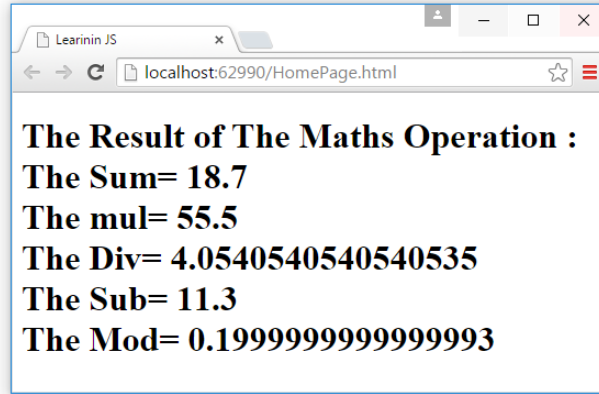
لاحظ كيف قُمنّا بتطبيق شيفرة لغة HTML وشيفرة لغة CSS على شيفرة لغة JavaScript وهذه إحدى مميزات لغة الجافا سكربت وهي إمكانية اقحام شيفرات لغتي HTML و CSS بداخل شيفرات هذه اللغة بشكل مباشر إلا إن لغة JavaScript تتعدى أكثر من إمكانية طباعة النصوص والتلاعب بها فهي لغة تتوفر على أغلب المفاهيم الموجودة في اللغات القوية وهذا يعني إنه سيكون بإمكاننا من خلال هذه اللغة القيام بحسابات رياضية معينة كُنّا عاجزين عن فعلها مسبقاً مع لغة HTML ولغة CSS حيث يمكن من خلال هذه اللغة تكوين المتغيرات variables او الدوال functions او المصفوفات Arrays او الكائنات Objects أو الحلقات التكرارية loops وغيرها من المفاهيم التي تُساعدنا في برمجة الصفحة بحيث تُصبح مُتفاعلة مع الزائر ولكي نتعرف على هذه المفاهيم بالتفصيل الدقيق تعال بنا نتعرف على أول مفهوم منها مع المتغيرات .

5 المتغيرات Variables

المتغيرات عبارة عن مواقع في الذاكرة العشوائية RAM تُخزن في هذه المواقع قيم معينة سواء أكانت قيم نصية string او رقمية Numbers بجميع انواعها صحيحة او نسبية سالبة او موجبة او حتى الرموز symbols يمكن تخزينها بشكل مؤقت في تلك المتغيرات وعندما نُعلن عن وجود متغير سواء أكان بلغة الجافا سكربت أو بأي لغة أخرى يجب أن تُراعي بعض الأمور منها أن لا يُترك هذا المتغير فارغ أي لا يجوز أن نُعلن عن وجود متغير ثم لا تستعمل هذا المتغير فهذا الأمر منطقياً خاطئ كذلك يجب أن تنتبه إلا إنه لا يجوز أن تستعمل المتغير قبل الإعلان عنه أيضاً لا يجوز استخدام بعض الكلمات كاسم للمتغير وهي الكلمات المحجوزة برمجياً مثل الكلمة if او else او var او for وغيرها من الكلمات كما يجب أن تُراعي إن الأحرف الكبيرة upper-case تختلف في آلية التعريف عن الأحرف الصغيرة lower-case فمثلاً إذا كان عندنا متغير أسمه Ali فهذا المتغير يختلف عن المتغير ali لمجرد الاختلاف بين الأحرف الكبيرة والصغيرة. ولكي نُعلن عن وجود متغير نستعمل الكلمة المحجوزة برمجياً var على سبيل المثال لو أعلنّا عن ثلاث متغيرات الأول يستعمل لخزن قيمة نصية string والثاني يستعمل لخزن قيمة رقمية صحيحة integer بينما المتغير الثالث يستعمل لخزن قيمة نسبية floating اي عدد نسبي (كسر) مع إجراء العمليات الرياضية الرئيسية (الجمع ، الطرح ، الضرب ، القسمة ، باقي القسمة) على القيمتين الرقميتين وطباعة الناتج لذا ادخل ملف لغة الجافا سكربت واكتب بداخله الشيفرة التالية:

```
var num1 = 15;
var num2 = 3.7;
var MyText = "The Result of The Maths Operation : ";
var r1 = num1 + num2;
var r2 = num1 * num2;
var r3 = num1 / num2;
var r4 = num1 - num2;
var r5 = num1 % num2;
document.writeln(MyText + "<br/>");
document.write("The Sum= " + r1 + "<br/>");
document.write("The mul= " + r2 + "<br/>");
document.write("The Div= " + r3 + "<br/>");
document.write("The Sub= " + r4 + "<br/>");
document.write("The Mod= " + r5 + "<br/>");
```

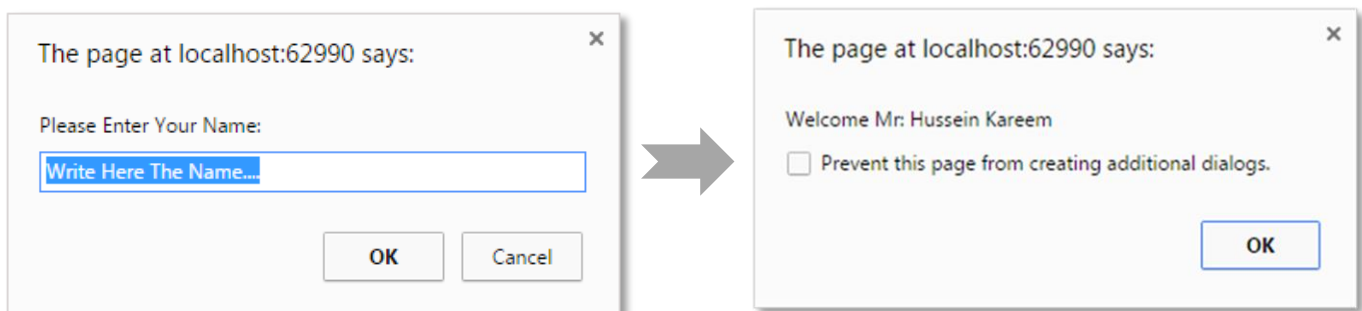
والنتيجة التي ستحصل عليها في المتصفح بعد حفظ المشروع وتنفيذ الصفحة :



هنا تأتي فائدة المتغيرات فكما تلاحظ بواسطتها أستطعنا إجراء العمليات الرياضية وهذا ما كان يستحيل علينا فعله بواسطة لغتي HTML و CSS حيث قمنا بالإعلان عن وجود متغيرين رقميين الأول خزننا به قيمة رقمية صحيحة وهو المتغير num1 بينما المتغير الثاني خزننا به قيمة رقمية نسبية وهو المتغير num2 ثم أعلننا عن وجود متغير نصي يحمل قيمة نصية وهو المتغير MyText بعدها أعلننا عن خمس متغيرات في كل متغير نُخزن ناتج عملية رياضية سيتم إجرائها على قيمة المتغيرين الرقميين ففي المتغير r1 قمنا بتخزين ناتج عملية الجمع بينما في المتغير r2 قمنا بتخزين ناتج عملية الضرب أما في المتغير r3 فقد قمنا بخزن ناتج عملية القسمة بينما في المتغير r4 فقد قمنا بخزن ناتج عملية الطرح وفي المتغير r5 قمنا بخزن ناتج عملية باقي القسمة أما في حالة لو أردنا إدخال قيم المتغيرات بمعنى نجعل المستخدم يدخل قيم المتغيرات دون تقييد عندها نستعمل الدالة prompt لغرض إدخال قيم تلك المتغيرات لأنك تلاحظ إننا أعطينا المتغيرات قيمها عندما أعلننا عنها في الكود البرمجي ولكن ماذا لو أردنا من الزائر ان يدخل القيم على سبيل المثال لو أردنا الزائر ان يدخل اسمه لغرض الترحيب به على اساس القيمة المدخلة والتي تمثل اسمه هنا يجب على المستخدم إدخال اسمه وبالتالي فإن القيمة التي سيدخلها المستخدم تُخزن في المتغير. لاحظ الشيفرة التالية حيث سنجعل المستخدم يدخل اسمه في نافذة الإدخال prompt وسيتم الترحيب به على اساس القيمة المدخلة :

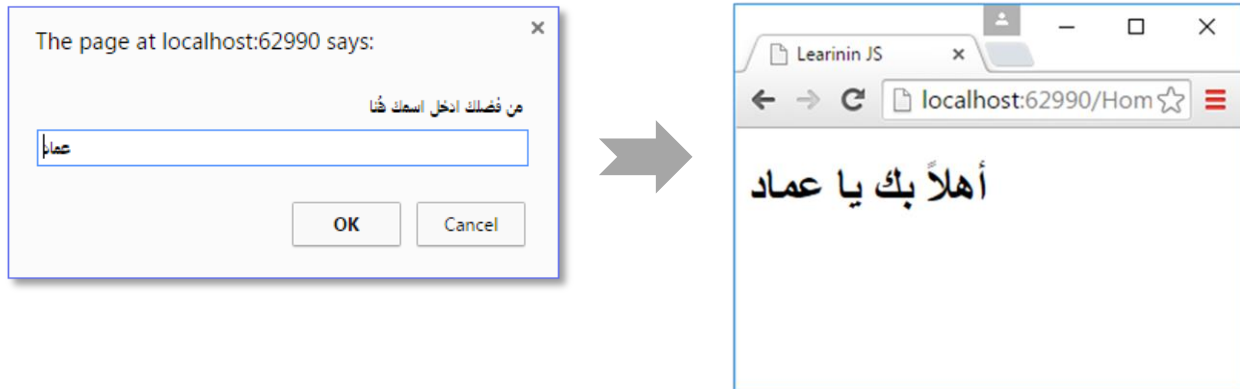
```
var name = window.prompt("Please Enter Your Name:", "Write Here The Name...");  
alert("Welcome Mr."+name);
```

والنتيجة هي ظهور نافذة الإدخال حيث سيقوم المستخدم بإدخال اسمه ويتم الترحيب به بطباعة اسمه مع رسالة ترحيبية في نافذة alert :



الآن تعال بنا نُنجز مثلاً آخر حيث سوف نكتب شيفرة تعمل على إخراج نافذة الإدخال prompt للزائر وعندما يقوم بإدخال اسمه يتم طباعة رسالة ترحيبية مع اسمه في المتصفح لذا ادخل ملف لغة الجافا سكربت واكتب الشيفرة التالية بداخله :

```
var name = window.prompt("من فضلك أدخل اسمك هنا", "... اسمك");  
document.write("أهلاً بك يا "+name);
```



5 البنية الشرطية

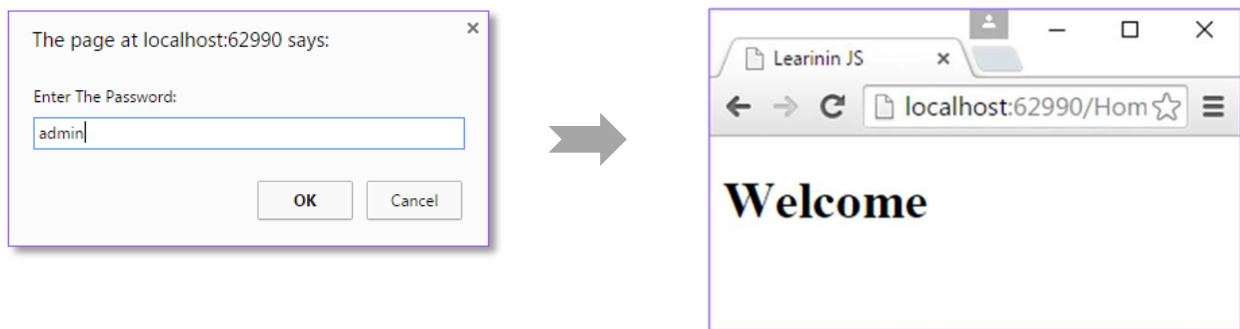
تُعتبر البنية الشرطية أحد مفاهيم البرمجة الأساسية والتي تعني إن هنالك مجموعة من الأوامر يتم تنفيذها وفق شرط مُعين حيث إما أن يكون هذا الشرط صحيح true بمعنى مُتحقق عندها سيتم تنفيذ الأوامر التابعة لجُملة الشرط وإما أن يكون هذا الشرط خاطئ false بمعنى غير مُتحقق عندها سيتم تنفيذ الأوامر الواقعة خارج جُملة الشرط وهذا الأسلوب بالتالي يُساعدنا في الانتقائية في تنفيذ الأوامر فكثيراً ما نحتاج إلى تنفيذ بعض الأوامر على أساس شرط مُعين والجُملة الشرطية في لغة الجافا سكربت على نوعين النوع الأول يكون باستخدام جُملة if else أما النوع الثاني يكون باستخدام جُملة switch إذ يتم استخدام النوع الأول عندما نريد أن يتم تنفيذ امر مُعين او مجموعة أوامر تابعة لجُملة if وفق شرط مُحدد فأن كان هذا الشرط صحيح يتم عندها تنفيذ كُل الأوامر الموجودة داخل جُملة if وإن كان الشرط خاطئ عندها يتم الانتقال إلى تنفيذ الأوامر الواقعة خارج جُملة if والتي تكون تابعة لجُملة else . تكون الصيغة البرمجية لجُملة if كالتالي:

```
if (شرط مُعين)  
{  
    // اوامر يتم تنفيذها عند تحقق الشرط  
}  
else  
{  
    // اوامر يتم تنفيذها عند فشل شرط جُملة if  
}
```

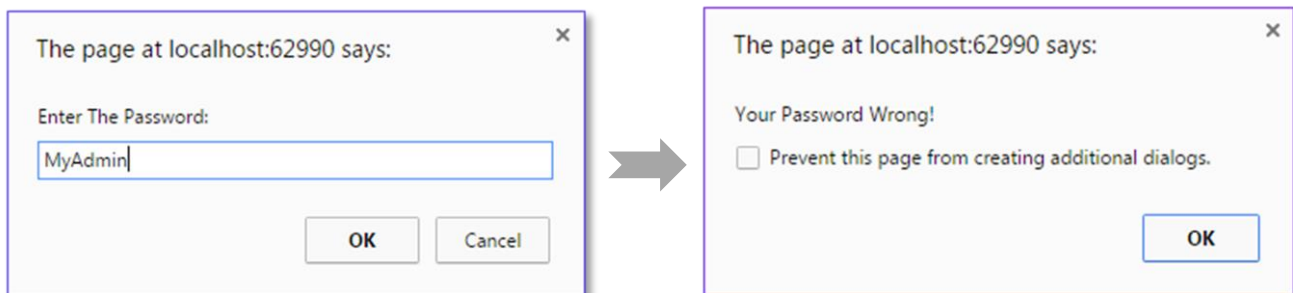
ولكي يتضح لنا هذا المفهوم تعال بنا نأخذ مثال عملي حيث سوف نجعل المُستخدم يُدخل كلمة سر password في نافذة الإدخال prompt وفي نفس الوقت لدينا كلمة سر محجوزة مُسبقاً في الكود البرمجي فإذا كانت كلمة السر التي سوف يُدخلها المُستخدم تُطابق كلمة السر المحجوزة في الكود البرمجي عندها سيتم طباعة كلمة Welcome للزائر دلالتاً على السماح له بدخول الصفحة أما إذا كانت كلمة السر التي يُدخلها لا تُطابق كلمة السر المحجوزة لدينا في الكود البرمجي عندها سيتم طباعة الرسالة Your Password Wrong! دلالتاً على أن المُستخدم غير مسموح له بدخول الصفحة لأن كلمة السر التي أدخلها خاطئة لذا ادخل ملف لغة الجافا سكربت وأكتب الشيفرة التالية بداخله :

```
var pass = "admin";
var user_password;
user_password = window.prompt("Enter The Password: ", "Password");
if (user_password == pass) {
    document.write("Welcome");
}
else {
    alert("Your Password Wrong!");
}
```

لاحظ إنه لدينا كلمة سر محجوزة مُسبقاً في المتغير pass وهي admin فإذا أدخل المُستخدم كلمة سر تُطابق تلك الكلمة عندها يُعتبر الشرط مُتحقق أي صحيح true وبالتالي يتم تنفيذ الأوامر التابعة لجملته ولدينا أمر طباعة الكلمة Welcome وبالتالي سيتم طباعة تلك الكلمة في المُتصفح في حال تطابق كلمتي السر بينما لو قام المُستخدم بأدخال كلمة سر لا تُطابق كلمة السر المحجوزة لدينا عندها سيعتبر الشرط خاطئ أي غير مُتحقق false وبالتالي يتم أهمله والانتقال لتنفيذ الأوامر التابعة لجمله else حيث سيتم طباعة الرسالة Your Password Wrong! دلالتاً على أن كلمة السر التي أدخلها المُستخدم غير صحيحة . لاحظ لو ادخل المُستخدم كلمة سر صحيحة :



أما في حال كانت كلمة السر غير صحيحة :



أما بالنسبة للنوع الثاني من جَمَل الشرط فيتمثل بجملة الشرط من نوع switch وتكون الصيغة البرمجية لهذا النوع كالتالي :

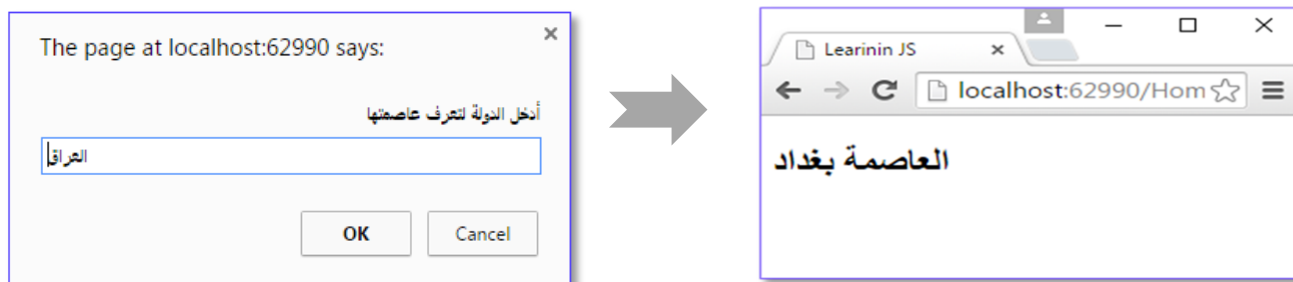
```
switch (هنا نضع المتغير الذي على اساس قيمته يتم تنفيذ الاحتمالات)
{
    case الاحتمال الأول : break; ; عملية مُعينة يتم تنفيذها في حالة كان المُدخل هو الاحتمال الأول :
    case الاحتمال الثاني : break; ; عملية مُعينة يتم تنفيذها في حالة كان المُدخل هو الاحتمال الثاني :
    case الاحتمال الثالث : break; ; عملية مُعينة يتم تنفيذها في حالة كان المُدخل هو الاحتمال الثالث :
    case الاحتمال الرابع : break; ; عملية مُعينة يتم تنفيذها في حالة كان المُدخل هو الاحتمال الرابع :
    .
    .
    .
    case الاحتمال الأخير : break; ; عملية مُعينة يتم تنفيذها في حالة كان المُدخل هو الاحتمال الأخير :
    default : break; ; عملية مُعينة يتم تنفيذها في حالة كان المُدخل ليس من الاحتمالات السابقة :
}
```

طبعاً مع الرؤية العملية ستفهم وتستوعب هذا النوع من جَمَل الشرط لذا تعال بنا نكتب شيفرة بسيطة بلغة الجافا سكربت نوضح ونفهم من خ لالها صيغة هذا النوع من جَمَل الشرط لذا ادخل ملف لغة الجافا سكربت واكتب الشيفرة التالية بداخله :

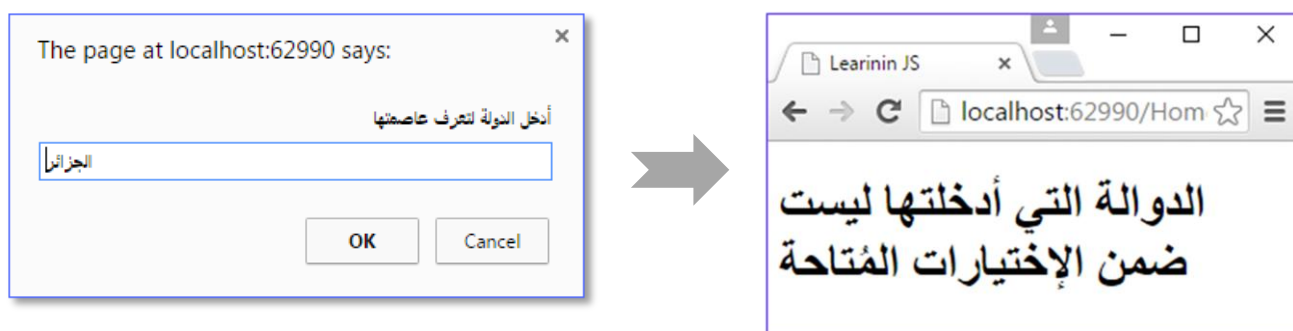
```
var country;
country = window.prompt( " , " أدخل الدولة لتعرف عاصمتها ");
switch (country) {
    case "العراق":
        document.writeln(" العاصمة بغداد "); break;
    case "سوريا":
        document.writeln(" العاصمة دمشق "); break;
    case "عُمان":
        document.writeln(" العاصمة مسقط "); break;
    case "البحرين":
        document.writeln(" العاصمة المنامة "); break;
    case "اليمن":
        document.writeln(" العاصمة صنعاء "); break;
    case "تونس":
        document.writeln(" العاصمة تونس "); break;
    case "مصر":
        document.writeln(" العاصمة القاهرة "); break;
    default: document.write( " الدولة التي أدخلتها ليست ضمن الإختيارات المُتاحة " );
}
```

لاحظ إن المُستخدم سيقوم بإدخال اسم دولة من الدول العربية فإذا قام بإدخال دولة من أصل مجموعة الدول المحجوزة لدينا في الكود البرمجي عندها سيتم طباعة عاصمة تلك الدولة لأن الشرط يكون قد تحقق عند هذه الحالة لكون الدولة التي أدخلها موجودة ضمن الإختيارات المُتاحة بينما في حالة قام المُستخدم بإدخال دولة ليست ضمن الإختيارات المُتاحة

سيعتبر الشرط غير مُتحقق وبالتالي ينتقل البرنامج لتنفيذ الأوامر التابعة لجُملة default . وعند التنفيذ وإدخال قيمة ضمن الإختيارات المُتاحة سنحصل على النتيجة التالية :



وعند مَحاولة إدخال قيمة ليست ضمن الإختيارات المُتاحة سنحصل على النتيجة التالية :



5 البنية التكرارية

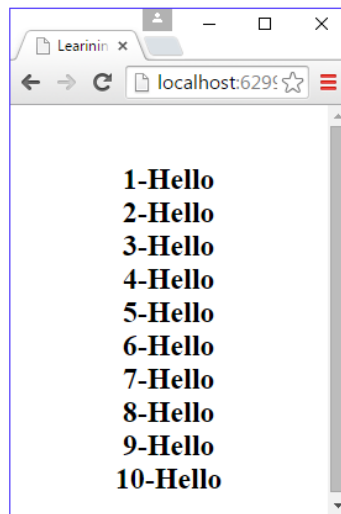
البنية التكرارية او كما تُسمى احياناً الحلقات التكرارية تُعتبر إحدى اهم مفاهيم البرمجة حيث يتم من خلالها تكرار خطوة مُعينة لأكثر من مرة لتحقيق غايات مُعينة كتجنب كتابة خطوات برمجية لأكثر من مرة او تكرار تنفيذ خطوة لأكثر من مرة إذ انه بواسطة هذه البنية يتم تكرار الخطوة المُراد تجنب كتابتها في كُل مرة لغرض توفير الوقت وتقليل حجم الكود وتسهيل الأمر على المُبرمج لأنه لن يكون مُضطراً لكتابة الخطوة التي يُريد تنفيذها لأكثر من مرة في كُل مرة اراد تنفيذها بل يضع حلقة تكرارية تُكرر له تلك الخطوة او العملية نياباً عن كتابتها برمجياً لأكثر من مرة فمثلاً لو اراد المُبرمج طباعة جُملة مُعينة 10 مرات في الحالة الإعتيادية كان ليُكرر أمر الطباعة 10 مرات لذا فبدل هذا العذاب البرمجي 😓 وضع حلقة تكرارية تُجنبك من تكرار كتابة أمر الطباعة بل هي تطبع لك الجُملة من خلال تكرار تنفيذها على عدد المرات التي ترغب به وكُل ما تحتاجه قيمة يبتدأ منها العد وقيمة يتوقف عندها العد ومقدار لزيادة او نُقصان العد. اعتقد ان غاية وهدف البنية التكرارية قد توضح بالنسبة لك خصوصاً بعد تخيل العذاب البرمجي 😂 حيث أن فكرتها الأساسية كما قلنا انها تُمكننا من تكرار مجموعة من الأوامر البرمجية التي لو كتبناها بالطريقة العادية لأرهقنا بمعنى إنها توفر علينا الجُهد والوقت والحلقات التكرارية على ثلاث انواع رئيسية النوع الأول البنية التكرارية باستخدام حلقة for التكرارية والنوع الثاني باستخدام حلقة while التكرارية والنوع الثالث باستخدام حلقة do التكرارية حيث أن كُل هذه الأنواع تتطلب قيمة ابتدائية للعمل وشرط توقف لأيقاف الحلقة التكرارية عن العمل عند تحقق الشرط ومقدار زيادة او نُقصان لحركة الحلقة

تدريجياً لأن الحلقة حينما تقوم بتكرار تنفيذ عمل مُعين تحتاج ان تدور على هذا العمل لأكثر من مرة هنا يتطلب الأمر لعمل هذه الحلقة مقدار زيادة او نقصان . ولكي نرى مفهوم البنية التكرارية دعنا نبدأ بأول نوع من أنواعها مع حلقة for التكرارية والتي تكن صيغتها البرمجية كالتالي :

(مقدار زيادة أو نقصان ; شرط توقف ; قيمة ابتدائية) **for**

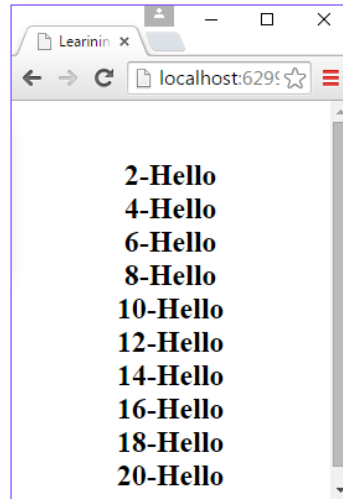
ويعتبر هذا النوع من الحلقات التكرارية الأكثر شيوعاً من حيث الاستخدام بالمقارنة مع حلقة while وحلقة do لأنه الأبسط لأن في هذا النوع تكون القيمة الابتدائية وشرط التوقف ومقدار الزيادة او النقصان كلها مُجمعة بين قوسي الحلقة التكرارية ولكي نستوعب هذا النوع أكثر دعنا نأخذ مثال عملي حيث سنقوم بتكرار طباعة كلمة Hello في المتصفح 10 مرات مع طباعة قيمة متغير الحلقة التكرارية لنحصل على تسلسل رقمي ليكون شكل الشيفرة ونتيجتها كالتالي :

```
for (i = 1; i <= 10; i++)
{
    document.write("<br/> "+i+"-Hello");
}
```



لاحظ إن الحلقة ابتدأت بالعد من الرقم 1 وانتهت بالعد عند الرقم 10 وكما حددنا لحظة إنشائها في الكود البرمجي وجعلناها تزداد في كل دورة لها بمقدار 1 ولهذا تظهر الأرقام متزايدة بمقدار 1 في كل دورة للحلقة التكرارية. ويمكنك بكل بساطة أن تتلاعب بمقدار الزيادة او بشرط التوقف او بالقيمة الابتدائية فمثلاً تستطيع أن تجعل مقدار الزيادة للحلقة بمقدار 2 وليبدأ العد من الرقم 2 ولتوقف عند الرقم 20 ليكون شكل الحلقة والنتيجة كالتالي :

```
for (i = 2; i <=20; i+=2)
{
    document.write("<br/> "+i+"-Hello");
}
```

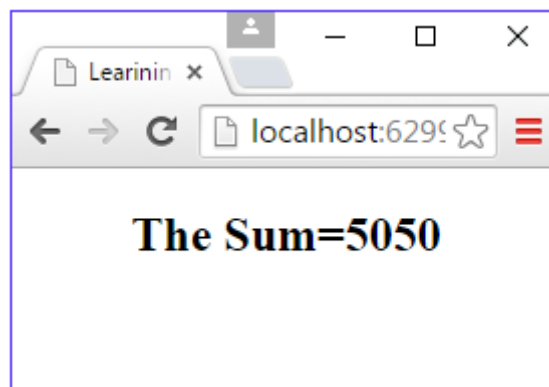


إذاً فالحلقات التكرارية ساعدتنا كثيراً في تجنب كتابة خطوة الطباعة في المثال السابق ولولا الحلقة التكرارية لأضطررنا لكتابة أمر الطباعة 10 مرات وبالتالي فقد قللنا وقلصنا حجم الكود البرمجي الذي كان ليصل إلى قرابة 11 سطر قلصناه في سطرين فقط وهذه هي إحدى فوائد الحلقات التكرارية فهي ضرورية وستحتاجها كثيراً ولكي نستوعب الفائدة الجلية من هذا المفهوم دعنا نأخذ مثال آخر على سبيل المثال لو اردنا جمع الأرقام من 1 إلى 100 في الحالة الاعتيادية وبدون استخدام الحلقات التكرارية كيف كنا سنفعل ذلك طبعاً كنا سنجمعها بالطريقة الاعتيادية :

$$1+2+3+4+.....100$$

وطبعاً هذا الأمر مُرهق بالنسبة للمبرمج ويشق عليه فعل ذلك لأن الكود سيكون ضخماً وبطيئاً في التحليل ولكن مع الحلقات التكرارية سيكون الكود البرمجي أبسط واسرع في التنفيذ والتحليل وشيفرة جمع الأرقام ستكون قصيرة بالمقارنة مع الطريقة العادية وكما تلاحظ:

```
var sum = 0;
for (i = 1; i <= 100; i++)
{
    sum = sum + i;
}
document.write("The Sum=" + sum);
```



أما النوع الثاني من الحلقات التكرارية فهو يتمثل بحلقة while وحلقة while تختلف عن حلقة for من حيث الهيكلية ولكنها تؤدي نفس العمل والصيغة البرمجية لهذه الحلقة تكون كالتالي :

القيمة الابتدائية

while (شرط التوقف)

{

/* جُملة الحلقة التكرارية حيث يُمكن ان تضم شتى انواع العمليات التي يُراد تكرار تنفيذها */

مُقدار الزيادة او النقصان

}

كما نلاحظ فحلقة while هيكليتها تختلف عن حلقة for فحلقة for كانت تجمع القيمة الابتدائية وشرط التوقف ومقدار الزيادة او النقصان بين قوسيهما بينما حلقة while لا تضم بين قوسيهما سوى شرط التوقف اما بالنسبة للقيمة الابتدائية فهي تقع خارج قوسي الحلقة ومقدار الزيادة او النقصان يقع داخل جُملة الحلقة ولكي نستوعب عمل هذا النوع من الحلقات التكرارية تعال بنا نكتب شيفرة بسيطة ولنرى نتيجتها في المتصفح عند تنفيذها ولنحاول إعادة كتابة الشيفرة السابقة التي قُمنا بواسطتها من جمع الأرقام من 1 إلى 100 ولكن هذه المرة باستخدام حلقة while حتى نستوعب هيكلية هذه الحلقة والفرق بينها وبين حلقة for السابقة من حيث الهيكلية لذا ادخل ملف لغة الجافا سكربت واكتب الشيفرة التالية بداخله :

```
var sum = 0;
var i = 1;
while (i <= 100)
{
    sum = sum + i;
    i++;
}
document.write("The Sum =" + sum);
```

